EVENT_INTEGRATION

Event Bus Integration Guide

Overview

The HelixCode event bus enables automatic notifications when system events occur. This guide shows how to integrate existing components with the event bus.

Quick Start

```
import (
    "dev.helix.code/internal/event"
    "dev.helix.code/internal/notification"
)

// Initialize event bus (usually done at application startup)
bus := event.GetGlobalBus()

// Create notification engine
notifEngine := notification.NewNotificationEngine()

// Register channels
slackChannel := notification.NewSlackChannel("https://
        hooks.slack.com/...", "#alerts", "HelixBot")
notifEngine.RegisterChannel(slackChannel)

// Create event handler
eventHandler :=
        notification.NewEventNotificationHandler(notifEngine)

// Register with event bus
eventHandler.RegisterWithEventBus(bus)

// Now publish events from your components
bus.Publish(context.Background(), event.Event{
    Type: event.EventTaskCompleted,
    Severity: event.SeverityInfo,
    Source: "task_manager",
    Data: map[string]interface{}{
        "task_id": "task-123",
```

```
        "duration": "2m30s",
    },
})
```

Integration Points

Task Manager

Add event publishing to task lifecycle methods:

```
// In internal/task/manager.go

import "dev.helix.code/internal/event"

func (m *Manager) CompleteTask(ctx context.Context, taskID
        string) error {
    // ... existing logic ...

    // Publish event
    bus := event.GetGlobalBus()
    bus.Publish(ctx, event.Event{
        Type: event.EventTaskCompleted,
        Severity: event.SeverityInfo,
        Source: "task_manager",
        TaskID: taskID,
        Data: map[string]interface{}{
            "task_id": taskID,
            "duration": duration.String(),
        },
    })

    return nil
}

func (m *Manager) FailTask(ctx context.Context, taskID string,
        err error) error {
    // ... existing logic ...

    bus := event.GetGlobalBus()
    bus.Publish(ctx, event.Event{
        Type: event.EventTaskFailed,
        Severity: event.SeverityError,
        Source: "task_manager",
        TaskID: taskID,
```

```
        Data: map[string]interface{}{
            "task_id": taskID,
            "error": err.Error(),
        },
    })

    return nil
}
```

Workflow Engine

```
// In internal/workflow/engine.go

func (e *Engine) CompleteWorkflow(ctx context.Context,
        workflowID string) error {
    // ... existing logic ...

    bus := event.GetGlobalBus()
    bus.Publish(ctx, event.Event{
        Type: event.EventWorkflowCompleted,
        Severity: event.SeverityInfo,
        Source: "workflow_engine",
        Data: map[string]interface{}{
            "workflow_id": workflowID,
            "workflow_name": workflow.Name,
        },
    })

    return nil
}
```

Worker Pool

```
// In internal/worker/pool.go

func (p *SSHWorkerPool) handleWorkerDisconnect(workerID string,
        reason error) {
    // ... existing logic ...

    bus := event.GetGlobalBus()
    bus.Publish(context.Background(), event.Event{
        Type: event.EventWorkerDisconnected,
        Severity: event.SeverityWarning,
        Source: "worker_pool",
        WorkerID: workerID,
```

```
        Data: map[string]interface{}{
            "worker_id": workerID,
            "host": worker.Host,
            "reason": reason.Error(),
        },
    })
}
```

Event Types Reference

See `internal/event/bus.go` for complete list of event types.

Task Events: - `EventTaskCreated`, `EventTaskAssigned`, `EventTaskStarted` - `EventTaskCompleted`, `EventTaskFailed`, `EventTaskPaused`

Workflow Events: - `EventWorkflowStarted`, `EventWorkflowCompleted`, `EventWorkflowFailed`

Worker Events: - `EventWorkerConnected`, `EventWorkerDisconnected`, `EventWorkerHealthDegraded`

System Events: - `EventSystemStartup`, `EventSystemShutdown`, `EventSystemError`

Testing

```
func TestMyComponent_EventPublishing(t *testing.T) {
    // Create test bus
    bus := event.NewEventBus(false)

    // Create capturer
    var capturedEvents []event.Event
    handler := func(ctx context.Context, evt event.Event) error {
        capturedEvents = append(capturedEvents, evt)
        return nil
    }

    bus.Subscribe(event.EventTaskCompleted, handler)

    // Test your component
    // ...

    // Verify event was published
    assert.Equal(t, 1, len(capturedEvents))
```

```
    assert.Equal(t, event.EventTaskCompleted,
        capturedEvents[0].Type)
}
```

Best Practices

1. **Always include context**: Use `ctx` parameter for cancellation
2. **Set severity appropriately**: Info/Warning/Error/Critical
3. **Include relevant IDs**: TaskID, WorkerID, ProjectID, UserID
4. **Add useful metadata**: Error messages, durations, statuses
5. **Use async bus for production**: `NewEventBus(true)` for better performance
6. **Handle errors gracefully**: Event publishing should not break core logic

Production Checklist

- [ ] Initialize global event bus at app startup
- [ ] Register notification handler with event bus
- [ ] Configure notification channels (Slack, Email, etc.)
- [ ] Add notification rules for different event types
- [ ] Integrate task manager with event bus
- [ ] Integrate workflow engine with event bus
- [ ] Integrate worker pool with event bus
- [ ] Add system startup/shutdown events
- [ ] Test event flow end-to-end
- [ ] Monitor event bus error logs